

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное учреждение  
высшего образования  
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии  
Департамент цифровых, робототехнических систем и электроники

**ОТЧЕТ  
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4  
ДИСЦИПЛИНЫ  
«ОСНОВЫ НЕЙРОННЫХ СЕТЕЙ»  
ВАРИАНТ № 2**

Выполнил:  
Левашев Тимур Рашидович  
3 курс, группа ИВТ-б-о-23-2,  
09.03.01 «Информатика и  
вычислительная техника»,  
направленность (профиль)  
«Программное обеспечение средств  
вычислительной техники и  
автоматизированных систем», очная  
форма обучения

---

(подпись)

Проверил:  
Доцент департамента цифровых,  
робототехнических систем и  
электроники института перспективной  
инженерии Воронкин Р.А

---

(подпись)

Отчет защищен с оценкой \_\_\_\_\_ Дата защиты \_\_\_\_\_

Ставрополь, 2026 г.

**Тема работы:** «Обработка текстов с помощью нейронных сетей»

**Цель работы:** подробно рассмотреть и освоить методы подготовки текстовых данных для подачи в нейронную сеть, а также научиться проектировать и обучать модели для решения задач классификации текстов (определение авторства или тональности отзывов)

## Порядок выполнения работы

Ссылка на Git репозиторий: [https://github.com/mazy99/neural-networks\\_4](https://github.com/mazy99/neural-networks_4)

### 1 Исследование влияния гиперпараметров полносвязной нейронной сети на точность классификации текстовых данных

#### 1.1 Загрузка датасета и распаковка архива с данными

##### Листинг 1.1 – Загрузка и распаковка датасета

```
# Загрузка датасета из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l7/writers.zip', None, quiet=True)
# Распаковка архива в папку writers
!unzip -qo writers.zip -d writers/

# Просмотр содержимого папки
!ls writers
```

```
'(Булгаков) Обучающая_5 вместе.txt'
'(Булгаков) Тестовая_2 вместе.txt'
'(Клиффорд Саймак) Обучающая_5 вместе.txt'
'(Клиффорд Саймак) Тестовая_2 вместе.txt'
'(Макс Фрай) Обучающая_5 вместе.txt'
'(Макс Фрай) Тестовая_2 вместе.txt'
'(О. Генри) Обучающая_50 вместе.txt'
'(О. Генри) Тестовая_20 вместе.txt'
'(Рэй Брэдберри) Обучающая_22 вместе.txt'
'(Рэй Брэдберри) Тестовая_8 вместе.txt'
'(Стругацкие) Обучающая_5 вместе.txt'
'(Стругацкие) Тестовая_2 вместе.txt'
```

Рисунок 1.1 – Результат загрузки датасета

## 1.2 Загрузка данных из файлов в обучающий и тестовый массивы

### Листинг 1.2 – Загрузка данных из файлов в массивы

```
# Загрузка датасета. Добавляются имена классов и соответствующие тексты.
# Все тексты преобразуются в строку и объединяются для каждого класса и выборки
CLASS_LIST = []
text_train = []
text_test = []

for file_name in os.listdir(FILE_DIR):
    # Выделение имени класса и типа выборки из имени файла
    m = re.match(r'\((.+)\) (\S+)_', file_name)
    # Если выделение получилось, то файл обрабатывается
    if m:
        class_name = m[1]
        subset_name = m[2].lower()
        # Проверка типа выборки в имени файла
        is_train = SIG_TRAIN in subset_name
        is_test = SIG_TEST in subset_name

        # Если тип выборки обучающая либо тестовая - файл обрабатывается
        if is_train or is_test:
            # Добавление нового класса, если его еще нет в списке
            if class_name not in CLASS_LIST:
                print(f'Добавление класса "{class_name}"')
                CLASS_LIST.append(class_name)
            # Инициализация соответствующих классу строк текста
            text_train.append('')
            text_test.append('')

            # Поиск индекса класса для добавления содержимого файла в выборку
            cls = CLASS_LIST.index(class_name)
            print(f'Добавление файла "{file_name}" в класс "{CLASS_LIST[cls]}"',
                  {subset_name} выборка.')
            with open(f'{FILE_DIR}/{file_name}', 'r') as f:
                # Загрузка содержимого файла в строку
                text = f.read()

            # Определение выборки, куда будет добавлено содержимое
            subset = text_train if is_train else text_test
            # Добавление текста к соответствующей выборке класса. Концы строк
            # заменяются на пробел
            subset[cls] += ' ' + text.replace('\n', ' ')

# Определение количества классов
CLASS_COUNT = len(CLASS_LIST)
```

```

Добавление класса "О. Генри"
Добавление файла "(О. Генри) Тестовая_20 вместе.txt" в класс "О. Генри", тестовая выборка.
Добавление класса "Рэй Брэдберри"
Добавление файла "(Рэй Брэдберри) Обучающая_22 вместе.txt" в класс "Рэй Брэдберри", обучающая выборка.
Добавление класса "Булгаков"
Добавление файла "(Булгаков) Обучающая_5 вместе.txt" в класс "Булгаков", обучающая выборка.
Добавление класса "Клиффорд Саймак"
Добавление файла "(Клиффорд Саймак) Тестовая_2 вместе.txt" в класс "Клиффорд Саймак", тестовая выборка.
Добавление файла "(Булгаков) Тестовая_2 вместе.txt" в класс "Булгаков", тестовая выборка.
Добавление класса "Макс Фрай"
Добавление файла "(Макс Фрай) Тестовая_2 вместе.txt" в класс "Макс Фрай", тестовая выборка.
Добавление класса "Стругацкие"
Добавление файла "(Стругацкие) Обучающая_5 вместе.txt" в класс "Стругацкие", обучающая выборка.
Добавление файла "(О. Генри) Обучающая_50 вместе.txt" в класс "О. Генри", обучающая выборка.
Добавление файла "(Рэй Брэдберри) Тестовая_8 вместе.txt" в класс "Рэй Брэдберри", тестовая выборка.
Добавление файла "(Стругацкие) Тестовая_2 вместе.txt" в класс "Стругацкие", тестовая выборка.
Добавление файла "(Клиффорд Саймак) Обучающая_5 вместе.txt" в класс "Клиффорд Саймак", обучающая выборка.
Добавление файла "(Макс Фрай) Обучающая_5 вместе.txt" в класс "Макс Фрай", обучающая выборка.

```

Рисунок 1.2 – Результат загрузки

### 1.3 Преобразование текстовых данных в цифровые векторы и создание частотного словаря

Листинг 1.3 – Тенизация слов

```

tokenizer = Tokenizer(
    num_words=VOCAB_SIZE,
    filters='!"#$%&()*+,-./:;<=>?@[\\]^_`{|}~«»\t\n\ха0\ufeff',
    lower=True, split=' ',
    oov_token='неизвестное_слово',
    char_level=False)

tokenizer.fit_on_texts(text_train)
items = list(tokenizer.word_index.items())

print(f"Первые 10 элементов словаря: {items[:10]}")

```

```

Первые 10 элементов словаря: [('неизвестное_слово', 1), ('и', 2), ('в', 3), ('не', 4), ('я', 5), ('что', 6), ('на', 7), ('с', 8), ('он', 9),

```

Рисунок 1.3 – Результат токенизации

### 1.4 Векторизация текстов, преобразование слов в последовательности числовых индексов

Листинг 1.4 – Векторизация текстов

```

train_seq = tokenizer.texts_to_sequences(text_train)

```

```
test_seq = tokenizer.texts_to_sequences(text_test)

print(f"Первые 10 слов в виде оригинального текста: {text_train[0][:68]}")
print(f"Первые 10 слов из первого текста обучающей выборки: {train_seq[0][:10]}")
print(f"Первые 10 слов в виде оригинального текста: {text_test[0][:69]}")
print(f"Первые 10 слов из первого текста тестовой выборки: {test_seq[0][:10]}")
```

```
Первые 10 слов в виде оригинального текста: «Лиса-на-рассвете» Коралио нежил в полуденном зное, как томная
Первые 10 слов из первого текста обучающей выборки: [4725, 7, 2449, 1757, 1, 3, 1, 1, 11, 1]
Первые 10 слов в виде оригинального текста: Багдадская птица Без всякого сомнения, дух и гений калифа Гаруна
Первые 10 слов из первого текста тестовой выборки: [1, 1501, 99, 1152, 1972, 1146, 2, 4039, 1, 1]
```

Рисунок 1.4 – Результат векторизации текста

## 1.5 Реализация алгоритма сегментации текстовых последовательностей методом скользящего окна

Листинг 1.5 – Нарезка текстовых векторов

```
def split_sequences(sequence: list, win_size: int = WIN_SIZE, win_hop: int = WIN_HOP) -> list:

    n = len(sequence)
    windows = []

    for i in range(0, n - win_size + 1, win_hop):
        window = sequence[i:i + win_size]

        if len(window) == win_size:
            windows.append(window)

    return windows
```

## 1.6 Формирование векторизованных выборок с учетом балансировки классов

Листинг 1.6 – Сопоставление фрагментов текста их идентификаторам класса

```
def vectorize_sequences(sequences: list, win_size: int = WIN_SIZE, win_hop: int = WIN_HOP) -> np.ndarray:

    class_count = len(sequences)

    x, y = [], []
```

```

for cls in range(class_count):

    vectors = split_sequences(sequences[cls], win_size, win_hop)

    vectors = vectors[:MAX_WINDOWS_PER_CLASS]

    x.extend(vectors)

    y.extend([cls] * len(vectors))
return np.array(x), np.array(y)

```

## 1.7 Разделение данных на обучающую и тестовую выборки

Листинг 1.7 – Разделение данных по выборкам

```

x_train, y_train = vectorize_sequences(train_seq)
x_test, y_test = vectorize_sequences(test_seq)

indices_train = np.arange(len(x_train))
indices_test = np.arange(len(x_test))

np.random.shuffle(indices_train)
np.random.shuffle(indices_test)

x_train = x_train[indices_train]
y_train = y_train[indices_train]

x_test = x_test[indices_test]
y_test = y_test[indices_test]

print(f'Размер обучающей выборки: {x_train.shape}, {y_train.shape}')
print(f'Размер тестовой выборки: {x_test.shape}, {y_test.shape}')

```

```

Размер обучающей выборки: (3537, 500), (3537,)
Размер тестовой выборки: (1346, 500), (1346,)

```

Рисунок 1.5 – Результат разделения данных

## 1.8 Преобразование текстовых последовательностей в разреженную матрицу методом Bag of Words (Мешок слов)

Листинг 1.8 – Преобразование текстов в матрицу BOW

```

x_train_01 = tokenizer.sequences_to_matrix(x_train.tolist())
x_test_01 = tokenizer.sequences_to_matrix(x_test.tolist())

# Вывод формы обучающей выборки в виде разреженной матрицы Bag of Words
print(x_train_01.shape)
# Вывод фрагмента отрезка обучающего текста в виде Bag of Words
print(x_train_01[0][0:100])

```

```

(3537, 20000)
[0.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  0.  1.  1.  1.  1.
 0.  0.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  0.  0.  1.  0.  1.  1.  1.  0.  1.  0.  1.  1.
 1.  1.  1.  1.  1.  1.  1.  1.  1.  1.  0.  1.  0.  1.  0.  0.  1.  1.  0.  0.  1.  1.  1.  1.  0.
 1.  0.  0.  1.  1.  1.  1.  0.  1.  0.  0.  0.  0.  1.  0.  0.  0.  0.  1.  0.  1.  0.  1.  0.  0.
 0.  1.  0.  0.]

```

Рисунок 1.6 – Результат преобразования

## 1.9 Создание функций обучения, оценки и визуализации нейронной сети

### Листинг 1.9 – Функции обучения, оценки и визуализации

```

# Функция компиляции и обучения модели нейронной сети
def compile_train_model(model,
                        x_train,
                        y_train,
                        x_val,
                        y_val,
                        optimizer='adam',
                        epochs=50,
                        batch_size=128,
                        figsize=(20, 5)):

    callback = tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        mode='min', min_delta=0.005,
        patience = 2,
        restore_best_weights=True
    )

    # Компиляция модели
    model.compile(optimizer=optimizer,
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy']
    )

```

```

# Вывод сводки
model.summary()

# Обучение модели с заданными параметрами
history = model.fit(x_train,
                    y_train,
                    epochs=epochs,
                    batch_size=batch_size,
                    validation_data=(x_val, y_val),
                    callbacks=[callback])

# Вывод графиков точности и ошибки
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=figsize)
fig.suptitle('График процесса обучения модели')
ax1.plot(history.history['accuracy'],
          label='Доля верных ответов на обучающем наборе')
ax1.plot(history.history['val_accuracy'],
          label='Доля верных ответов на проверочном наборе')
ax1.xaxis.get_major_locator().set_params(integer=True)
ax1.set_xlabel('Эпоха обучения')
ax1.set_ylabel('Доля верных ответов')
ax1.legend()

ax2.plot(history.history['loss'],
          label='Ошибка на обучающем наборе')
ax2.plot(history.history['val_loss'],
          label='Ошибка на проверочном наборе')
ax2.xaxis.get_major_locator().set_params(integer=True)
ax2.set_xlabel('Эпоха обучения')
ax2.set_ylabel('Ошибка')
ax2.legend()
plt.show()

# Функция вывода результатов оценки модели на заданных данных
def eval_model(model, x, y_true, class_labels=[], title='', figsize=(10, 10)):

    y_pred = model.predict(x)

    y_pred_cls = np.argmax(y_pred, axis=1)
    y_true_cls = y_true
    acc = (y_pred_cls == y_true_cls).mean()

    cm = confusion_matrix(y_true_cls, y_pred_cls, normalize='true')

    cm_rounded = np.around(cm, 3)

    fig, ax = plt.subplots(figsize=figsize)
    ax.set_title(f'Confusion Matrix: {title}', fontsize=16)

```



```

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm_rounded,
    display_labels=class_labels
)

disp.plot(ax=ax, cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.show()

print('-' * 80)
print(f'Accuracy: {acc:.4f}')

# правильная интерпретация
for i in range(len(class_labels)):
    predicted_class = np.argmax(cm[i])
    status = "OK" if predicted_class == i else "ERROR"

    print(f"{class_labels[i]:<20} → {class_labels[predicted_class]:<20} | {status}")

print(f"\nMean class accuracy: {np.mean(np.diag(cm)):.4f}")

# Совместная функция обучения и оценки модели нейронной сети
def compile_train_eval_model(model,
                             x_train,
                             y_train,
                             x_test,
                             y_test,
                             class_labels=CLASS_LIST,
                             title='',
                             optimizer='adam',
                             epochs=50,
                             batch_size=128,
                             graph_size=(20, 5),
                             cm_size=(10, 10)):

    # Компиляция и обучение модели на заданных параметрах
    # В качестве проверочных используются тестовые данные
    compile_train_model(model,
                        x_train, y_train,
                        x_test, y_test,
                        optimizer=optimizer,
                        epochs=epochs,
                        batch_size=batch_size,
                        figsize=graph_size)

    # Вывод результатов оценки работы модели на тестовых данных
    eval_model(model, x_test, y_test,
               class_labels=class_labels,

```

```
title=title,  
figsize=cm_size)
```

## 1.10 Создание модели нейронной сети, использующую BOW

### Листинг 1.10 – Создание модели нейронной сети

```
model_text_bow_softmax = Sequential()  
model_text_bow_softmax.add(Dense(100, input_dim=VOCAB_SIZE, activation="relu",  
kernel_regularizer=tf.keras.regularizers.L2(  
    l2=0.001  
)))  
model_text_bow_softmax.add(BatchNormalization())  
model_text_bow_softmax.add(Dropout(0.5))  
model_text_bow_softmax.add(Dense(50, activation="relu"))  
model_text_bow_softmax.add(BatchNormalization())  
model_text_bow_softmax.add(Dropout(0.2))  
model_text_bow_softmax.add(Dense(CLASS_COUNT, activation='softmax'))  
  
compile_train_eval_model(model_text_bow_softmax,  
    x_train_01, y_train,  
    x_test_01, y_test,  
    class_labels=CLASS_LIST,  
    title='Bow')
```

Model: sequential\_15

| Layer (type)                                | Output Shape | Param #   |
|---------------------------------------------|--------------|-----------|
| dense_38 (Dense)                            | (None, 100)  | 2,000,100 |
| batch_normalization_23 (BatchNormalization) | (None, 100)  | 400       |
| dropout_23 (Dropout)                        | (None, 100)  | 0         |
| dense_39 (Dense)                            | (None, 50)   | 5,050     |
| batch_normalization_24 (BatchNormalization) | (None, 50)   | 200       |
| dropout_24 (Dropout)                        | (None, 50)   | 0         |
| dense_40 (Dense)                            | (None, 6)    | 306       |

Рисунок 1.7 – Структура модели

```

Epoch 1/50
28/28 ————— 8s 164ms/step - accuracy: 0.7009 - loss: 1.0675 - val_accuracy: 0.5981 - val_loss: 1.5336
Epoch 2/50
28/28 ————— 1s 20ms/step - accuracy: 0.9627 - loss: 0.3887 - val_accuracy: 0.6924 - val_loss: 1.2707
Epoch 3/50
28/28 ————— 0s 15ms/step - accuracy: 0.9912 - loss: 0.2793 - val_accuracy: 0.7779 - val_loss: 1.0784
Epoch 4/50
28/28 ————— 0s 16ms/step - accuracy: 0.9975 - loss: 0.2306 - val_accuracy: 0.8046 - val_loss: 0.9596
Epoch 5/50
28/28 ————— 0s 15ms/step - accuracy: 0.9989 - loss: 0.2058 - val_accuracy: 0.8262 - val_loss: 0.8694
Epoch 6/50
28/28 ————— 1s 15ms/step - accuracy: 0.9994 - loss: 0.1866 - val_accuracy: 0.8410 - val_loss: 0.7878
Epoch 7/50
28/28 ————— 0s 16ms/step - accuracy: 0.9994 - loss: 0.1713 - val_accuracy: 0.8477 - val_loss: 0.7293
Epoch 8/50
28/28 ————— 1s 15ms/step - accuracy: 0.9994 - loss: 0.1586 - val_accuracy: 0.8484 - val_loss: 0.6873
Epoch 9/50
28/28 ————— 0s 16ms/step - accuracy: 0.9997 - loss: 0.1459 - val_accuracy: 0.8484 - val_loss: 0.6522
Epoch 10/50
28/28 ————— 0s 15ms/step - accuracy: 1.0000 - loss: 0.1340 - val_accuracy: 0.8499 - val_loss: 0.6238
Epoch 11/50
28/28 ————— 0s 15ms/step - accuracy: 0.9997 - loss: 0.1241 - val_accuracy: 0.8529 - val_loss: 0.5999
Epoch 12/50
28/28 ————— 0s 15ms/step - accuracy: 0.9997 - loss: 0.1143 - val_accuracy: 0.8470 - val_loss: 0.5749
Epoch 13/50
...
Epoch 17/50
28/28 ————— 0s 14ms/step - accuracy: 1.0000 - loss: 0.0778 - val_accuracy: 0.8418 - val_loss: 0.5406
Epoch 18/50
28/28 ————— 0s 14ms/step - accuracy: 0.9997 - loss: 0.0727 - val_accuracy: 0.8328 - val_loss: 0.5420
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Рисунок 1.8 – Результат обучения модели

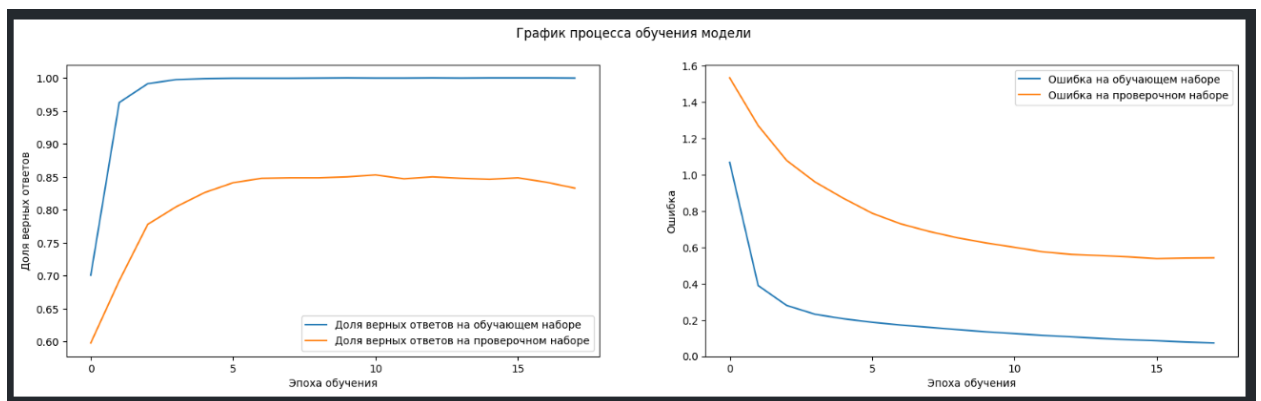


Рисунок 1.9 – Визуализация обучения модели

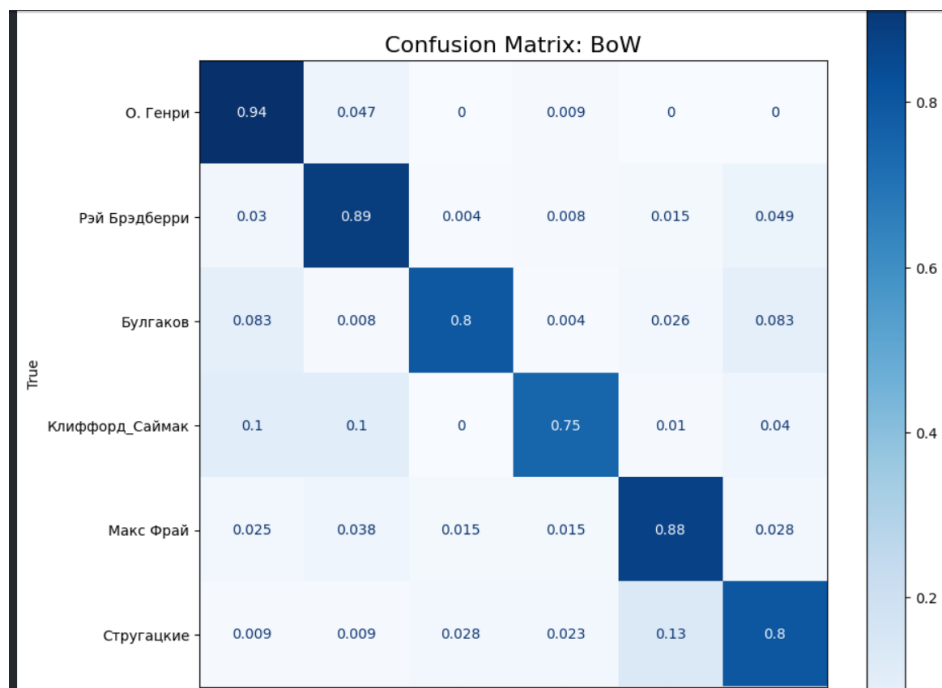


Рисунок 1.10 – Матрица ошибок

### 1.11 Преобразование данных в наборы последовательностей фиксированной длины для подачи в нейронную сеть

Листинг 1.11 – Формирование обучающих последовательностей для векторного представления (Embedding)

```
# 1. Переводим тексты в длинные последовательности индексов слов
train_seq_raw = tokenizer.texts_to_sequences(text_train)
test_seq_raw = tokenizer.texts_to_sequences(text_test)

# 2. Нарезаем их на окна с помощью твоих функций
# x_train_02 будет формой (Кол-во_окон, WIN_SIZE)
# y_train_02 будет формой (Кол-во_окон,) – это просто ID классов
x_train_02, y_train_02 = vectorize_sequences(train_seq_raw)
x_test_02, y_test_02 = vectorize_sequences(test_seq_raw)

# Проверяем размерности
print(f'X обучающий: {x_train_02.shape}') # Например, (3537, 500)
print(f'Y обучающий: {y_train_02.shape}') # Должно быть (3537,)
```

## 1.12 Создание модели и оценка ее точности

### Листинг 1.12 – Модель нейронной сети

```
# Архитектура со слоем Embedding и регуляризацией
model_text_emb_20 = Sequential()
model_text_emb_20.add(Embedding(input_dim = VOCAB_SIZE, output_dim = 50,
input_length=WIN_SIZE))

model_text_emb_20.add(SpatialDropout1D(0.5))

model_text_emb_20.add(GlobalAveragePooling1D())

model_text_emb_20.add(Dense(32,
activation="relu",
kernel_regularizer=tf.keras.regularizers.L2(l2=0.001)
))

model_text_emb_20.add(BatchNormalization())
model_text_emb_20.add(Dropout(0.5))

model_text_emb_20.add(Dense(CLASS_COUNT, activation='softmax'))

# Входные данные подаются в виде последовательностей индексов,
# а не векторов bag of words
compile_train_eval_model(model_text_emb_20,
x_train_02, y_train_02,
x_test_02, y_test_02,
class_labels=CLASS_LIST,
title='Embedding/20')
```

```
Epoch 1/50
28/28 — 7s 125ms/step - accuracy: 0.4326 - loss: 1.5679 - val_accuracy: 0.2920 - val_loss: 1.7883
Epoch 2/50
28/28 — 0s 7ms/step - accuracy: 0.7246 - loss: 1.0634 - val_accuracy: 0.2920 - val_loss: 1.7531
Epoch 3/50
28/28 — 0s 7ms/step - accuracy: 0.8779 - loss: 0.6721 - val_accuracy: 0.2920 - val_loss: 1.7214
Epoch 4/50
28/28 — 0s 7ms/step - accuracy: 0.9406 - loss: 0.4360 - val_accuracy: 0.2920 - val_loss: 1.6842
Epoch 5/50
28/28 — 0s 8ms/step - accuracy: 0.9714 - loss: 0.3020 - val_accuracy: 0.2927 - val_loss: 1.6455
Epoch 6/50
28/28 — 0s 7ms/step - accuracy: 0.9822 - loss: 0.2240 - val_accuracy: 0.2927 - val_loss: 1.6032
Epoch 7/50
28/28 — 0s 7ms/step - accuracy: 0.9895 - loss: 0.1789 - val_accuracy: 0.3031 - val_loss: 1.5559
Epoch 8/50
28/28 — 0s 7ms/step - accuracy: 0.9926 - loss: 0.1513 - val_accuracy: 0.3076 - val_loss: 1.5083
Epoch 9/50
28/28 — 0s 7ms/step - accuracy: 0.9946 - loss: 0.1272 - val_accuracy: 0.3618 - val_loss: 1.4441
Epoch 10/50
28/28 — 0s 7ms/step - accuracy: 0.9952 - loss: 0.1121 - val_accuracy: 0.3863 - val_loss: 1.3868
Epoch 11/50
28/28 — 0s 7ms/step - accuracy: 0.9966 - loss: 0.1025 - val_accuracy: 0.4703 - val_loss: 1.3309
Epoch 12/50
28/28 — 0s 7ms/step - accuracy: 0.9966 - loss: 0.0949 - val_accuracy: 0.5401 - val_loss: 1.2566
Epoch 13/50
...
Epoch 26/50
28/28 — 0s 7ms/step - accuracy: 0.9997 - loss: 0.0460 - val_accuracy: 0.7816 - val_loss: 0.6793
Epoch 27/50
28/28 — 0s 7ms/step - accuracy: 0.9980 - loss: 0.0447 - val_accuracy: 0.7920 - val_loss: 0.6670
```

Рисунок 1.12 – Результат обучения модели

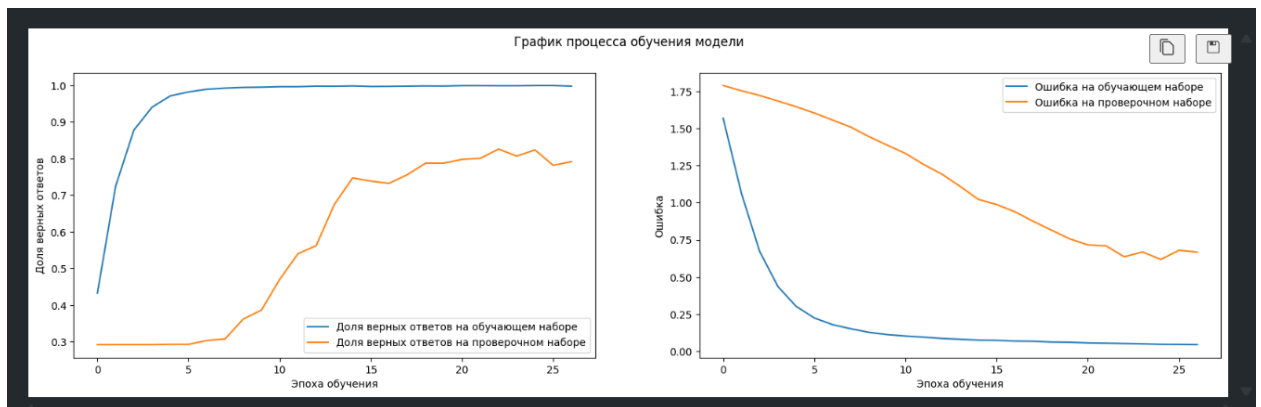


Рисунок 1.13 – График обучения модели

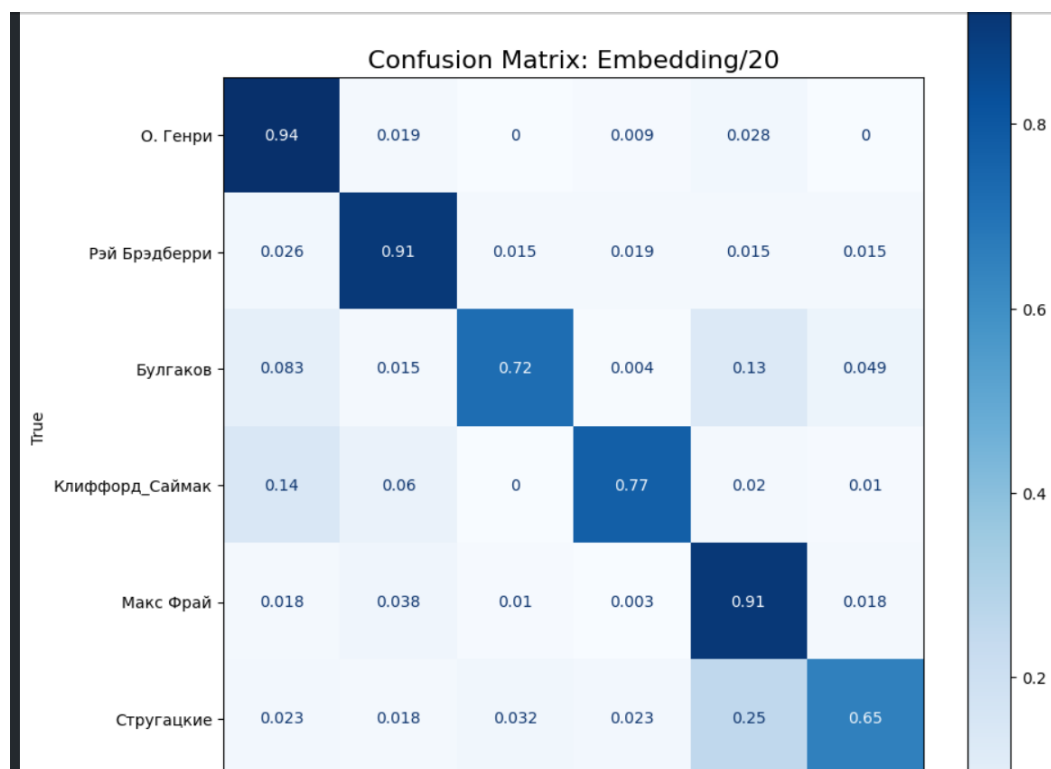


Рисунок 1.14 – Матрица ошибок модели

## 2 Решение задачи анализа тональности (Sentiment Analysis) на примере классификации отзывов пользователей

### 2.1 Формирование набора данных из текстовых файлов отзывов

Листинг 2.1 – Загрузка и разметка текстовых данных (позитивные и негативные отзывы)

```

data = []

with open(os.path.join(DATA_DIR, "Негативный отзыв.txt"), 'r', encoding='utf-8')
as f:
    neg_texts = f.readlines()

for t in neg_texts:
    data.append({
        'text': t.strip(),
        'label': 0
    })

    with open(os.path.join(DATA_DIR, "Позитивный отзыв.txt"), 'r', encoding='utf-
8') as f:
        pos_texts = f.readlines()

for t in pos_texts:
    data.append({
        'text': t.strip(),
        'label': 1
    })

df = pd.DataFrame(data)

print(df.shape)
display(df)

```

(3379, 2)

|      | text                                              | label |
|------|---------------------------------------------------|-------|
| 0    | После 170 на трассе она чувствует себя неувере... | 0     |
| 1    | Кстати сидения очень удобными тоже не назовёшь... | 0     |
| 2    | Шляпа, а не авто, все не продумано до мелочей.... | 0     |
| 3    | Прям фу фу фу и дизайн снаружи и внутри. Прода... | 0     |
| 4    | , негодование обычно возникает из-за несостояв... | 0     |
| ...  | ...                                               | ...   |
| 3374 | Тесла летает как ракета                           | 1     |
| 3375 | космос                                            | 1     |
| 3376 | КРУТОТЕНЬ 🍌🍌🍌🤩🤩🤩                                  | 1     |
| 3377 | космомобиль                                       | 1     |
| 3378 | офигенная тачка!                                  | 1     |

3379 rows × 2 columns

Рисунок 2.1 – Наборы данных

## 2.2 Предобработка текстовых данных и лемматизация

Листинг 2.2 – Функция очистки текста и приведения слов к нормальной форме

```
morph = pymorphy3.MorphAnalyzer()

def clean_text(text):
    text = re.sub(r'^а-яА-Яа-зА-З', ' ', text.lower())
    # Лемматизация каждого слова
    words = [morph.parse(word)[0].normal_form for word in text.split()]
    return ' '.join(words)
```

|                       | text                                              | label |
|-----------------------|---------------------------------------------------|-------|
| 0                     | после на трасса она чувствовать себя неуверенно   | 0     |
| 1                     | кстати сидение очень удобный тоже не назов шь ... | 0     |
| 2                     | шляпа а не авто всё не продумать до мелочь пок... | 0     |
| 3                     | прям фу фу фу и дизайн снаружи и внутри продат... | 0     |
| 4                     | негодование обычно возникать из за несостоявши... | 0     |
| ...                   | ...                                               | ...   |
| 3374                  | тесло летать как ракета                           | 1     |
| 3375                  | космос                                            | 1     |
| 3376                  | крутотень                                         | 1     |
| 3377                  | космомобиль                                       | 1     |
| 3378                  | офигенный тачка                                   | 1     |
| 3379 rows x 2 columns |                                                   |       |

Рисунок 2.2 – Результат очистки текста

## 2.3 Балансировка классов обучающей выборки

Листинг 2.3 – Выравнивание количества объектов по классам методом случайной выборки

```
min_len = df['label'].value_counts().min()

df_balanced = df.groupby('label').sample(min_len, random_state=42)
display(df_balanced['label'].value_counts())
display(df['label'].value_counts())
```



## 2.4 Разделение данных на обучающую и тестовую выборки

Листинг 2.4 – Разделение данных на выборки

```
X_train, X_test, y_train, y_test = train_test_split(
    df_balanced['text'],
    df_balanced['label'],

    test_size=0.2,
    random_state=42,
    stratify=df_balanced['label']
)
print(X_train)
```

## 2.5 Векторизация текстовых данных с использованием TF-IDF и n-грамм

Листинг 2.5 – Преобразование текстов в числовые признаки с помощью слоя TextVectorization

```
vectorize_layer = TextVectorization(
    max_tokens=VOCAB_SIZE,
    output_mode='tf_idf',
    ngrams=(1, 2)
)

vectorize_layer.adapt(X_train)

X_train_vec = vectorize_layer(X_train)
X_test_vec = vectorize_layer(X_test)
```

## 2.6 Построение и обучение модели классификации на основе Bag-of-Words

Листинг 2.6 – Реализация нейронной сети для классификации текстов

```
model_bow = Sequential([
    Dense(128, activation='relu', input_shape=(VOCAB_SIZE,)),
    Dropout(0.5),
```

```

BatchNormalization(),
Dense(64, activation='relu'),
Dropout(0.3),
BatchNormalization(),
Dense(1, activation='sigmoid')
])

compile_train_eval_model(model_bow,
                          X_train_vec, y_train,
                          X_test_vec, y_test,
                          class_labels=CLASS_LIST,
                          title='Bag of Words')

```

| Layer (type)                                | Output Shape | Param # |
|---------------------------------------------|--------------|---------|
| dense_72 (Dense)                            | (None, 128)  | 640,128 |
| dropout_45 (Dropout)                        | (None, 128)  | 0       |
| batch_normalization_38 (BatchNormalization) | (None, 128)  | 512     |
| dense_73 (Dense)                            | (None, 64)   | 8,256   |
| dropout_46 (Dropout)                        | (None, 64)   | 0       |
| batch_normalization_39 (BatchNormalization) | (None, 64)   | 256     |
| dense_74 (Dense)                            | (None, 1)    | 65      |

Рисунок 2.3 – Структура модели

```

Epoch 1/50
119/119 ————— 6s 23ms/step - accuracy: 0.5563 - loss: 0.7976 - val_accuracy: 0.6429 - val_loss: 0.6354
Epoch 2/50
119/119 ————— 0s 4ms/step - accuracy: 0.6626 - loss: 0.6146 - val_accuracy: 0.6891 - val_loss: 0.5984
Epoch 3/50
119/119 ————— 0s 4ms/step - accuracy: 0.7695 - loss: 0.4714 - val_accuracy: 0.7227 - val_loss: 0.5586
Epoch 4/50
119/119 ————— 0s 4ms/step - accuracy: 0.8232 - loss: 0.3859 - val_accuracy: 0.7311 - val_loss: 0.5138
Epoch 5/50
119/119 ————— 0s 4ms/step - accuracy: 0.8589 - loss: 0.3326 - val_accuracy: 0.7227 - val_loss: 0.6208
Epoch 6/50
119/119 ————— 1s 4ms/step - accuracy: 0.8832 - loss: 0.2743 - val_accuracy: 0.7710 - val_loss: 0.4942
Epoch 7/50
119/119 ————— 0s 4ms/step - accuracy: 0.8953 - loss: 0.2417 - val_accuracy: 0.7374 - val_loss: 0.5487
Epoch 8/50
119/119 ————— 1s 6ms/step - accuracy: 0.9116 - loss: 0.2262 - val_accuracy: 0.7311 - val_loss: 0.6414
Epoch 9/50
119/119 ————— 1s 5ms/step - accuracy: 0.9268 - loss: 0.1853 - val_accuracy: 0.7668 - val_loss: 0.5380
Epoch 10/50
119/119 ————— 1s 4ms/step - accuracy: 0.9311 - loss: 0.1785 - val_accuracy: 0.7626 - val_loss: 0.5856
Epoch 11/50
119/119 ————— 0s 4ms/step - accuracy: 0.9342 - loss: 0.1785 - val_accuracy: 0.7521 - val_loss: 0.5329

```

Рисунок 2.4 – Результат обучения модели

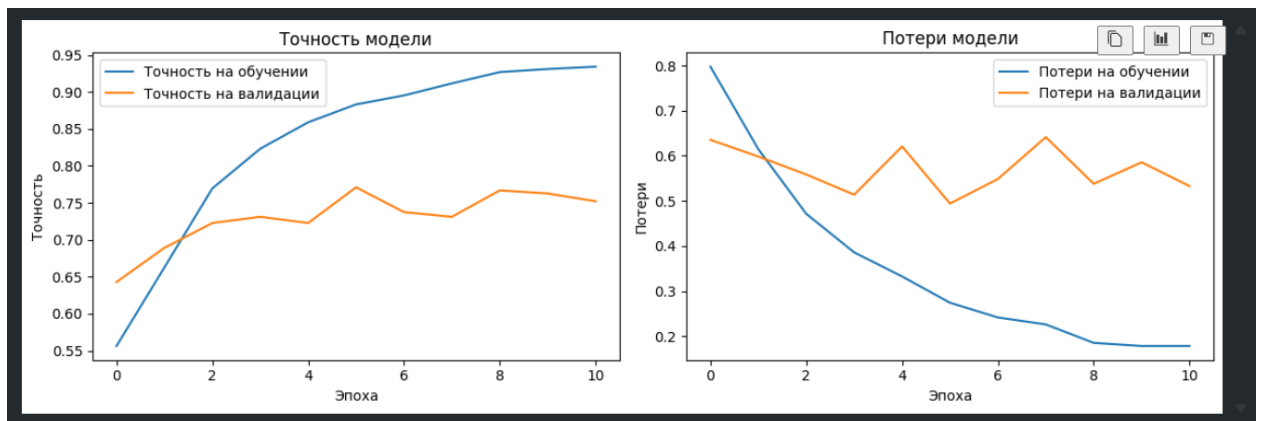


Рисунок 2.5 – График обучения и тестирования модели

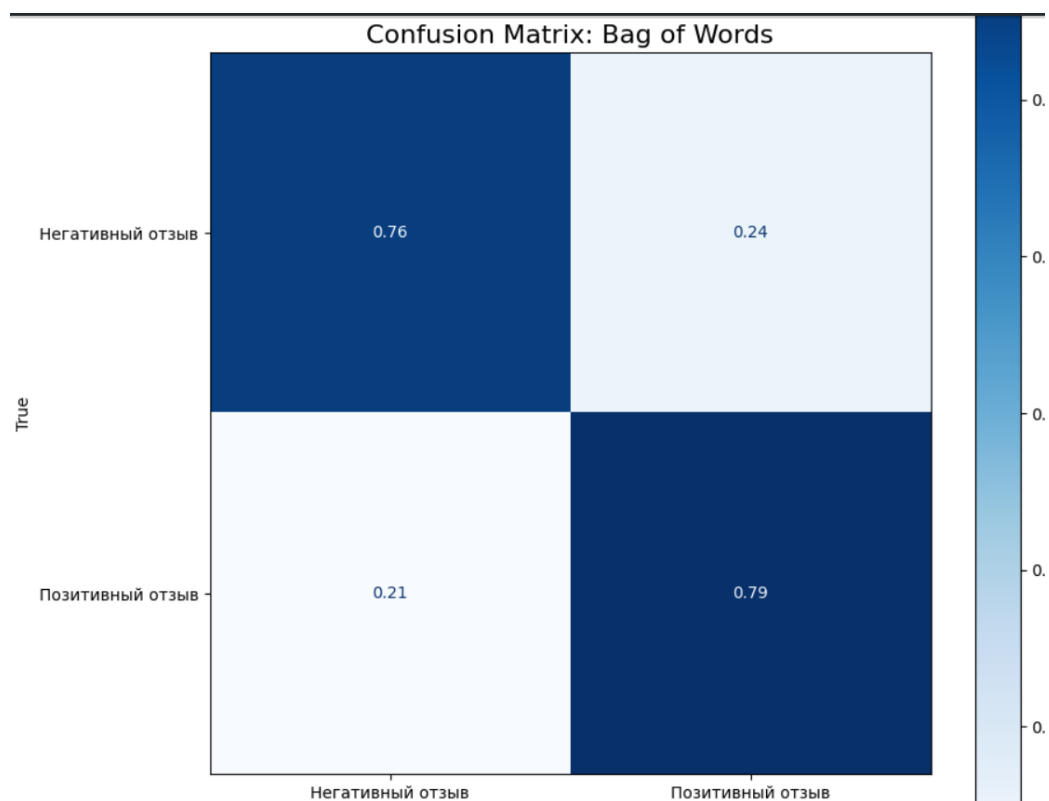


Рисунок 2.6 – Матрица ошибок модели

```

-----
Accuracy: 0.7710
Негативный отзыв    → 0.7563
Позитивный отзыв    → 0.7857

Mean class accuracy: 0.7710

<keras.src.callbacks.history.History at 0x7d8441f6ce90>

```

Рисунок 2.7 – Точность по каждому классу

### 3 Сравнительный анализ эффективности моделей Bag of Words и Embedding в задаче многоклассовой классификации текстов 20 авторов

#### 3.1 Слайсинг текстов с перекрытием

##### Листинг 3.1 – Загрузка и чанкинг текстов

```
data = []

for file_name in os.listdir(FILE_DIR):
    if not file_name.endswith('.txt'):
        continue

    author = file_name.replace('.txt', '')
    file_path = os.path.join(FILE_DIR, file_name)

    with open(file_path, 'r', encoding='utf-8') as f:
        text = f.read().replace('\n', ' ')

    for i in range(0, len(text) - WIN_SIZE, WIN_HOP):
        chunk = text[i:i + WIN_SIZE]

        data.append({
            'author': author,
            'text': chunk
        })

df = pd.DataFrame(data)

print("Размер:", df.shape)
print(df.columns)
display(df['author'].value_counts())
```

|             | count |
|-------------|-------|
| author      |       |
| Чехов       | 13221 |
| Достоевский | 10355 |
| Пушкин      | 6814  |
| Шолохов     | 6770  |
| Толстой     | 6712  |
| Гончаров    | 6207  |
| Пастернак   | 6176  |
| Васильев    | 5903  |
| Катаев      | 5156  |
| Горький     | 5044  |
| Носов       | 4796  |
| Куприн      | 4655  |
| Беляев      | 4508  |
| Лесков      | 4302  |
| Булгаков    | 4000  |
| Каверин     | 3983  |
| Тургенев    | 3958  |
| Лермонтов   | 3943  |
| Гоголь      | 3928  |
| Грибоедов   | 1936  |

Рисунок 3.1 – Результат нарезки текста

### 3.2 Балансировка классов методом андерсемплинга

Листинг 3.2 – Балансировка датасета

```
MIN_SAMPLES = df['author'].value_counts().min()

df_balanced = (
    df.groupby('author')
    .apply(lambda x: x.sample(MIN_SAMPLES, random_state=42))
    .reset_index(drop=True)
)

df_balanced['author'].value_counts()
```

| author      | count |
|-------------|-------|
| Беляев      | 1936  |
| Булгаков    | 1936  |
| Васильев    | 1936  |
| Гоголь      | 1936  |
| Гончаров    | 1936  |
| Горький     | 1936  |
| Грибоедов   | 1936  |
| Достоевский | 1936  |
| Каверин     | 1936  |
| Катаев      | 1936  |
| Куприн      | 1936  |
| Лермонтов   | 1936  |
| Лесков      | 1936  |
| Носов       | 1936  |
| Пастернак   | 1936  |
| Пушкин      | 1936  |
| Толстой     | 1936  |
| Тургенев    | 1936  |
| Чехов       | 1936  |
| Шолохов     | 1936  |

Рисунок 3.2 – Результат балансировки датасета

### 3.3 Разделение данные на обучающую и тестовую выборки

Листинг 3.3 – Разделение датасета на выборки

```
train_df, test_df = train_test_split(
    df_balanced,
    test_size=0.2,
    random_state=42,
    stratify=df_balanced['author'])

display(train_df['author'].value_counts())
display(test_df['author'].value_counts())
```

| author      | count |
|-------------|-------|
| Булгаков    | 1549  |
| Лесков      | 1549  |
| Катаев      | 1549  |
| Пушкин      | 1549  |
| Гончаров    | 1549  |
| Горький     | 1549  |
| Тургенев    | 1549  |
| Чехов       | 1549  |
| Беляев      | 1549  |
| Лермонтов   | 1549  |
| Каверин     | 1549  |
| Гоголь      | 1549  |
| Толстой     | 1549  |
| Шолохов     | 1549  |
| Носов       | 1549  |
| Васильев    | 1549  |
| Куприн      | 1548  |
| Достоевский | 1548  |
| Грибоедов   | 1548  |
| Пастернак   | 1548  |

Рисунок 3.3 – Обучающая выборка

| author      | count |
|-------------|-------|
| Пастернак   | 388   |
| Достоевский | 388   |
| Грибоедов   | 388   |
| Куприн      | 388   |
| Беляев      | 387   |
| Катаев      | 387   |
| Лесков      | 387   |
| Гоголь      | 387   |
| Пушкин      | 387   |
| Шолохов     | 387   |
| Васильев    | 387   |
| Горький     | 387   |
| Гончаров    | 387   |
| Толстой     | 387   |
| Каверин     | 387   |
| Чехов       | 387   |
| Носов       | 387   |
| Тургенев    | 387   |
| Лермонтов   | 387   |
| Булгаков    | 387   |

Рисунок 3.4 – Тестовая выборка

### 3.4 Кодирование категориальных меток (авторов текстов)

#### Листинг 3.4 – Кодирование категориальных меток

```
le = LabelEncoder()

y_train = le.fit_transform(train_df['author'])
y_test = le.transform(test_df['author'])
```

### 3.5 Тенизация текстов и построение словаря

#### Листинг 3.5 – Тенизация текстов

```
t = Tokenizer(
    num_words=VOCAB_SIZE,
    filters='!"#$%&()*+,-./...:;<=>?@[\\]^_`{|}~«»\t\n\xa0\ufe0f',
    lower=True, split=' ',
    oov_token='неизвестное_слово',
    char_level=False)
t.fit_on_texts(train_df['text'])
```

### 3.6 Преобразование текстов в бинарные векторы

#### Листинг 3.6 – Преобразование текстов в векторы

```
x_train = t.texts_to_matrix(train_df['text'], mode='binary')
x_test = t.texts_to_matrix(test_df['text'], mode='binary')
```

### 3.7 Создание и обучение нейронной сети для Bag-of-Words

#### Листинг 3.7 – Создание модели

```
model_bow = Sequential([
    Dense(128, activation='relu', input_shape=(VOCAB_SIZE,)),
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(len(le.classes_), activation='softmax')
])

compile_train_eval_model(model_bow,
                          x_train, y_train,
```



```
x_test, y_test,
class_labels=le.classes_,
title='Bag of Words')
```

| Layer (type)        | Output Shape | Param #   |
|---------------------|--------------|-----------|
| dense_6 (Dense)     | (None, 128)  | 1,280,128 |
| dropout_4 (Dropout) | (None, 128)  | 0         |
| dense_7 (Dense)     | (None, 64)   | 8,256     |
| dropout_5 (Dropout) | (None, 64)   | 0         |
| dense_8 (Dense)     | (None, 20)   | 1,300     |

Рисунок 3.5 – Структура модели

```
Epoch 1/50
968/968 — 7s 5ms/step - accuracy: 0.6141 - loss: 1.3139 - val_accuracy: 0.9393 - val_loss: 0.2557
Epoch 2/50
968/968 — 7s 7ms/step - accuracy: 0.8963 - loss: 0.3467 - val_accuracy: 0.9504 - val_loss: 0.1702
Epoch 3/50
968/968 — 5s 5ms/step - accuracy: 0.9410 - loss: 0.1942 - val_accuracy: 0.9540 - val_loss: 0.1474
Epoch 4/50
968/968 — 5s 5ms/step - accuracy: 0.9550 - loss: 0.1452 - val_accuracy: 0.9566 - val_loss: 0.1431
Epoch 5/50
968/968 — 5s 5ms/step - accuracy: 0.9629 - loss: 0.1176 - val_accuracy: 0.9557 - val_loss: 0.1422
Epoch 6/50
968/968 — 5s 5ms/step - accuracy: 0.9671 - loss: 0.1044 - val_accuracy: 0.9565 - val_loss: 0.1467
Epoch 7/50
968/968 — 11s 11ms/step - accuracy: 0.9703 - loss: 0.0959 - val_accuracy: 0.9574 - val_loss: 0.1437
Epoch 8/50
968/968 — 4s 4ms/step - accuracy: 0.9717 - loss: 0.0893 - val_accuracy: 0.9579 - val_loss: 0.1466
Epoch 9/50
968/968 — 4s 4ms/step - accuracy: 0.9720 - loss: 0.0857 - val_accuracy: 0.9578 - val_loss: 0.1494
Epoch 10/50
968/968 — 6s 6ms/step - accuracy: 0.9760 - loss: 0.0751 - val_accuracy: 0.9595 - val_loss: 0.1477
242/242 — 1s 3ms/step
```

Рисунок 3.6 – Результат обучения модели

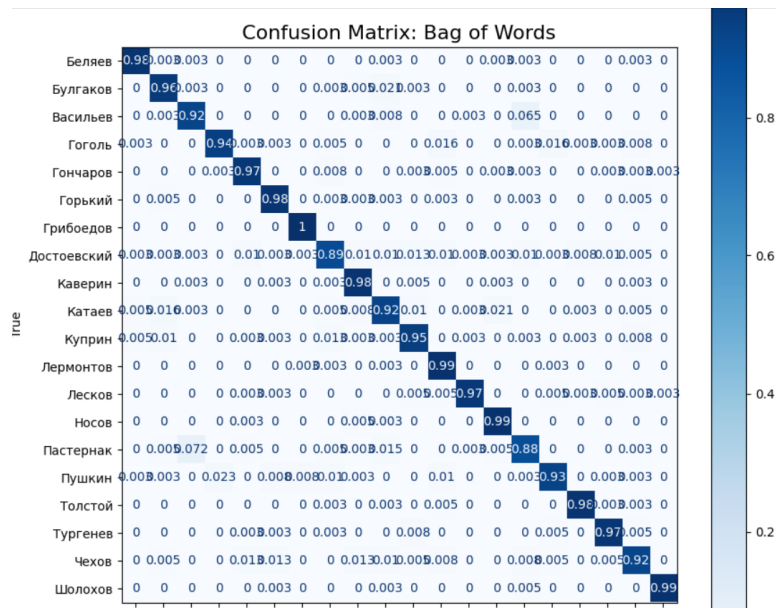


Рисунок 3.7 – Матрица ошибок модели

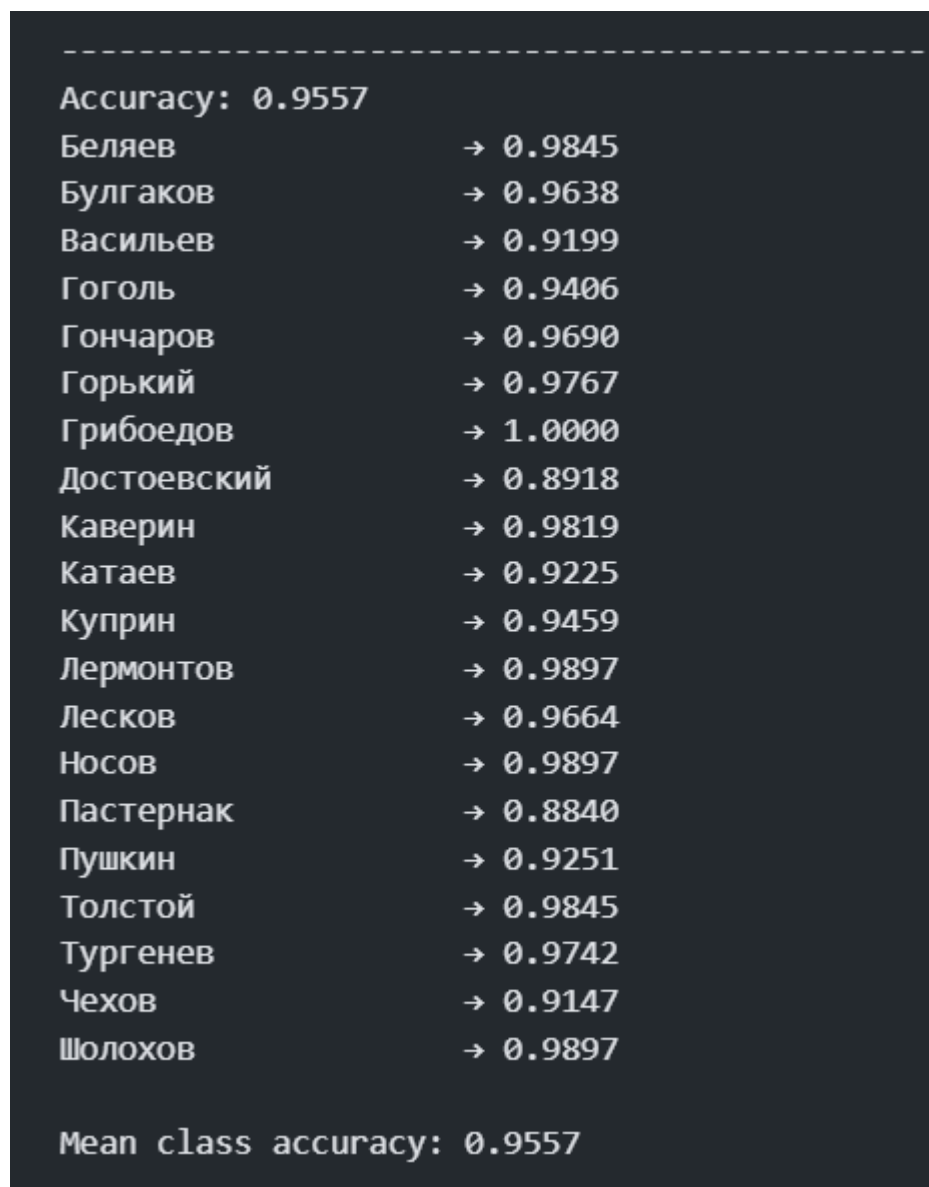


Рисунок 3.8 – Точность по каждому классу

### 3.8 Преобразование текстов в числовые последовательности с паддингом для эмбединга

Листинг 3.8 – Перевод текста в последовательности чисел

```
X_train_seq = t.texts_to_sequences(train_df['text'])
X_test_seq = t.texts_to_sequences(test_df['text'])

X_train = pad_sequences(X_train_seq, maxlen=MAX_LEN, padding='post',
                        truncating='post')
X_test = pad_sequences(X_test_seq, maxlen=MAX_LEN, padding='post',
                       truncating='post')
```

### 3.9 Построение модели с обучаемыми эмбедингами

Листинг 3.9 – Создание модели

```
model_emb = Sequential([
    Embedding(
        input_dim=VOCAB_SIZE,
        output_dim=128,
        input_length=MAX_LEN
    ),

    SpatialDropout1D(0.3),

    GlobalAveragePooling1D(),

    Dense(128, activation='relu'),
    Dropout(0.5),

    Dense(len(le.classes_), activation='softmax')
])

compile_train_eval_model(model_emb,
                          X_train, y_train,
                          X_test, y_test,
                          class_labels=le.classes_,
                          title='EMB')
```

енерировать + Код + Markdown + Colab + Выполнить все + Очистить все выходные данные + Структура + Выбрат

```

Epoch 1/50
968/968 — 8s 5ms/step - accuracy: 0.1005 - loss: 2.8949 - val_accuracy: 0.2450 - val_loss: 2.6010
Epoch 2/50
968/968 — 4s 4ms/step - accuracy: 0.3562 - loss: 2.1265 - val_accuracy: 0.5837 - val_loss: 1.5231
Epoch 3/50
968/968 — 5s 5ms/step - accuracy: 0.6161 - loss: 1.2686 - val_accuracy: 0.7429 - val_loss: 0.8975
Epoch 4/50
968/968 — 4s 4ms/step - accuracy: 0.7409 - loss: 0.8525 - val_accuracy: 0.8004 - val_loss: 0.6599
Epoch 5/50
968/968 — 4s 4ms/step - accuracy: 0.8078 - loss: 0.6259 - val_accuracy: 0.8440 - val_loss: 0.5189
Epoch 6/50
968/968 — 5s 5ms/step - accuracy: 0.8462 - loss: 0.4957 - val_accuracy: 0.8724 - val_loss: 0.4336
Epoch 7/50
968/968 — 4s 4ms/step - accuracy: 0.8728 - loss: 0.4073 - val_accuracy: 0.8811 - val_loss: 0.3718
Epoch 8/50
968/968 — 4s 4ms/step - accuracy: 0.8969 - loss: 0.3328 - val_accuracy: 0.8642 - val_loss: 0.4297
Epoch 9/50
968/968 — 6s 5ms/step - accuracy: 0.9104 - loss: 0.2850 - val_accuracy: 0.9026 - val_loss: 0.3105
Epoch 10/50
968/968 — 4s 4ms/step - accuracy: 0.9245 - loss: 0.2415 - val_accuracy: 0.8963 - val_loss: 0.3381
Epoch 11/50
968/968 — 4s 4ms/step - accuracy: 0.9306 - loss: 0.2221 - val_accuracy: 0.9223 - val_loss: 0.2497
Epoch 12/50
968/968 — 5s 5ms/step - accuracy: 0.9400 - loss: 0.1914 - val_accuracy: 0.9042 - val_loss: 0.2992
Epoch 13/50
...
968/968 — 4s 5ms/step - accuracy: 0.9666 - loss: 0.0969 - val_accuracy: 0.9390 - val_loss: 0.2097
Epoch 24/50
968/968 — 4s 5ms/step - accuracy: 0.9740 - loss: 0.0792 - val_accuracy: 0.9332 - val_loss: 0.2250
242/242 — 1s 2ms/step
  
```

Рисунок 3.9 – Результат обучения модели

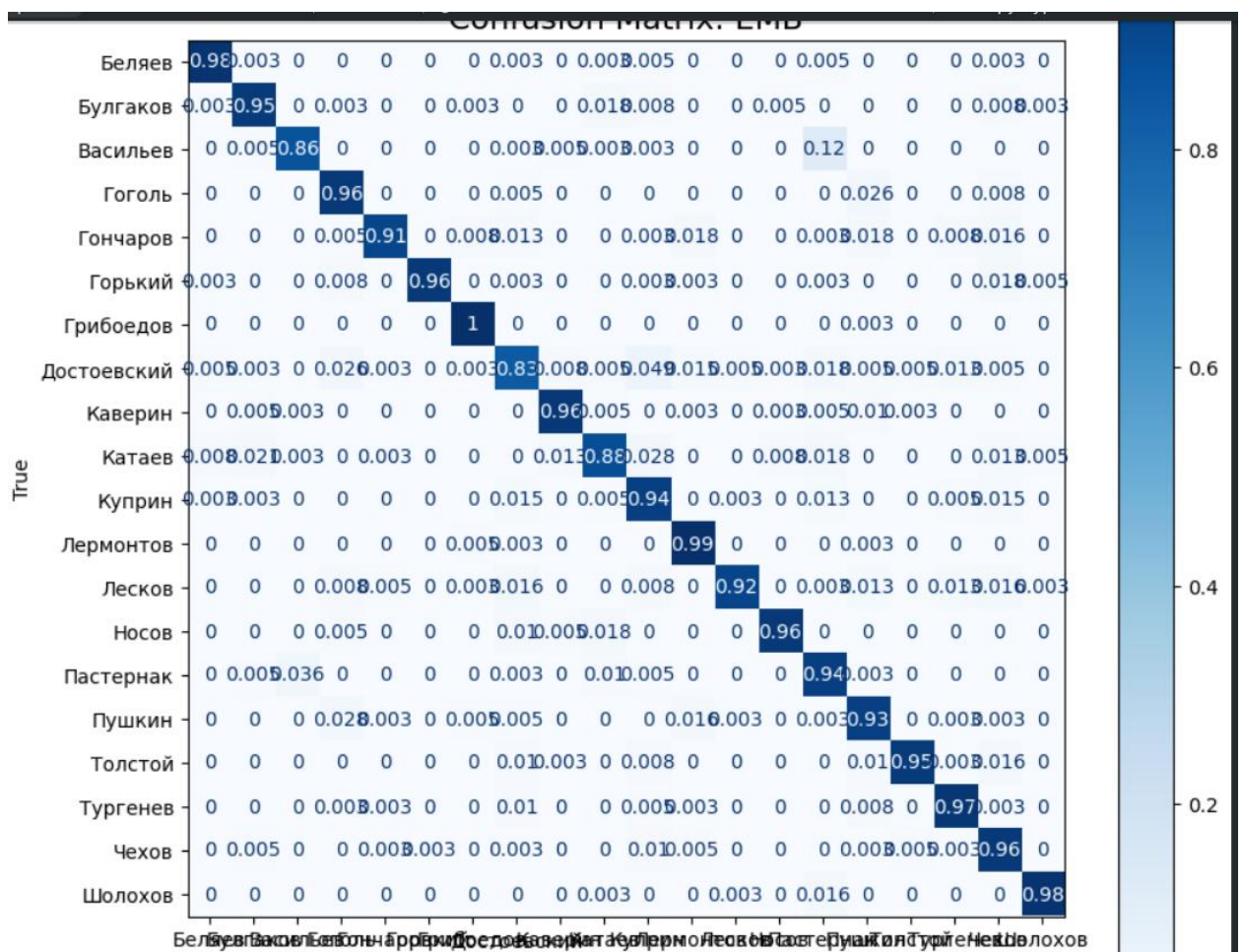


Рисунок 3.10 – Матрица ошибок

```
-----
Ассигасу: 0.9412
Беляев          → 0.9793
Булгаков        → 0.9509
Васильев        → 0.8630
Гоголь          → 0.9612
Гончаров        → 0.9096
Горький         → 0.9561
Грибоедов       → 0.9974
Достоевский     → 0.8299
Каверин         → 0.9638
Катаев          → 0.8811
Куприн          → 0.9381
Лермонтов       → 0.9897
Лесков          → 0.9147
Носов           → 0.9612
Пастернак       → 0.9381
Пушкин          → 0.9328
Толстой         → 0.9509
Тургенев        → 0.9664
Чехов           → 0.9612
Шолохов         → 0.9793
```

```
Mean class accuracy: 0.9413
```

Рисунок 3.11 – Точность модели по каждому классу